# Практическая работа 4. Работа с векторами. Математические функции

В языке C (в частности, и в диалекте CUDA C) векторы обычно представляются массивами, как правило, динамическими (память выделяется с помощью функций семейства malloc() и освобождается вызовом free(), аналоги в CUDA см. ниже).

В функции же передается не сам массив целиком, а лишь *указатель* на его начало (переменная фактически целочисленного типа, размер которого зависит от архитектуры (как правило, 32 или 64 разряда), содержащая адрес начала массива). Зная размер одного элемента, легко вычислить адрес любого элемента по его смещению.

## Сложение векторов

Давайте вспомним программу, которая генерирует два вектора целых чисел, копирует их в память GPU, выполняет сложение на GPU, результат копирует обратно в память CPU. GPU обычно не работает напрямую с памятью CPU, хотя это технически возможно. Для начала необходимо выделить память под данные на device, для этого используется фукция cudaMalloc(), после этого копируем данные из памяти host в память device с помощью cudaMemcpy(). Далее для каждого элемента вектора запускается отдельный процесс сложения, т. е. в данном случае – 128 нитей. На CPU это могло быть накладно, но GPU как раз рассчитан на работу с большим количеством потоков, и всё происходит достаточно быстро. После того, как операция завершена с помощью cudaMemcpy(), копируем результат обратно на host и освобождаем память device с помощью cudaFree(). В тексте ядра есть переменная threadIdx, это системная переменная CUDA, содержащая номер нити.

Приведем целиком программу сложения двух векторов с помощью технологии CUDA.

```
#define N 128 //количество элементов массива
// Ядро
__global__ void add( int *a, int *b, int *c )
```

```
{
    //вычисление индекса элемента
    int tid = threadIdx.x;
    //проверка на выход за пределы массива
    if(tid > N-1) return;
    //поэлементное сложение массивов
    c[tid] = a[tid] + b[tid];
}

int main() {
    // выделение памяти под массивы на CPU
    int host_a[N], host_b[N], host_c[N];
    // выделение памяти под массивы для копирования
    // на GPU
    int *dev_a, *dev_b, *dev_c;
    // заполнение массивов
    for (int i=0; i<N; i++)
    {
      host_a[i] = i * i;
        host_b[i] = - i;
    }
    // выделение памяти под массивы на GPU
    cudaMalloc( (void**)&dev_a, N * sizeof(int) );
    cudaMalloc( (void**)&dev_b, N * sizeof(int) );
    cudaMalloc( (void**)&dev_c, N * sizeof(int) );

    // копирование данных в память GPU
    cudaMemcpy( dev_a, host_a, N * sizeof(int),
    cudaMemcpyHostToDevice ) ;
    cudaMemcpy( dev_b, host_b, N * sizeof(int),
    cudaMemcpyHostToDevice ) ;

    // вызов ядра
    add<<<1, N>>>( dev_a, dev_b, dev_c );

    // получаем результат расчета
    cudaMemcpy( host_c, dev_c, N * sizeof(int),
    cudaMemcpyDeviceToHost ) ;
```

```
    // вывод результатов
    for (int i=0; i<N; i++)
    {
        printf( "%d + %d = %d\n", host_a[i],
        host_b[i], host_c[i] );
    }

    // освобождение памяти
    cudaFree( dev_a );
    cudaFree( dev_b );
    cudaFree( dev_c );
    return 0;
}
```

**Вычисление математических функций**

В CUDA существуют аналоги ряда математических функций, включенных в стандартную библиотеку языка Си (<math.h>). Так, например, прототип CUDA-версии синуса (sinf) имеет вид

```
__device__ float __sinf(float x);
```

Перечислим основные различия:

– функции CUDA вычисляются непосредственно на GPU;

– большинство функций работают только с float-аргументами (GPU, как правило, в основном работают с float, точности достаточно для основных задач, возлагаемых на GPU);

– имеют модификатор __device__, вследствие чего могут вызываться только с GPU (т.е. из kernel-функций);

Рассмотрим следующую задачу:

1. Выделить на GPU массив из n = $10^9$ элементов типа float.

2. Инициализировать их в ядре следующим образом:

```
arr[i] = sin((i % 360) * pi / 180)
```

где `pi = 3.141592653`.

3. Скопировать массив в память CPU, посчитать ошибку

```
err = sum_i(abs(sin((i % 360) * pi / 180) - arr[i]))
/ n
```

4. Провести исследование при использовании массива типа double.

Основной частью данной задачи является написание ядра, в котором будет инициализироваться массив (и вызываться синус).

```
#define BASE_TYPE float
#define MAX_GRIDSIZE 1

__global__ void sinMass(BASE_TYPE *A, int arraySize)
{
    int index = (blockIdx.y * MAX_GRIDSIZE + blockIdx.x)
    * blockDim.x + threadIdx.x;
  if (index < arraySize)
        [index] = sin((BASE_TYPE)((index % 360) * M_PI /
        180));

    // другие варианты вычисления синуса
    // A[index] = sinf((index % 360) * M_PI / 180);
    // A[index] = __sinf((index % 360) * M_PI
    // / 180);
}
```

Приведем список встроенных математических функций для работы с типом float:

| **Функция** | **Операция** |
|---|---|
| `__fadd_[rn,rz,ru,rd](x,y),` | x + y |
| `__fsub_[rn,rz,ru,rd](x,y),` | x – y |
| `__fmul_[rn,rz,ru,rd](x,y),` | x * y |
| `__fmaf_[rn,rz,ru,rd](x,y,z)` | x * y + z |

| | |
|---|---|
| `__frcp_[rn,rz,ru,rd](x)` | 1 / x |
| `__fsqrt_[rn,rz,ru,rd](x)` | $\sqrt{x}$ |
| `__frsqrt_rn(x)` | $1 / \sqrt{x}$ |
| `__fdiv_[rn,rz,ru,rd](x,y)` | x / y |
| `__fdividef(x,y)` | x / y; при $y>2^{126}$ результат = 0 |
| `__expf(x)` | $e^x$ |
| `__exp10f(x)` | $10^x$ |
| `__logf(x)` | ln x |
| `__log2f(x)` | $\log_2 x$ |
| `__log10f(x)` | lg x |
| `__sinf(x)` | sin x |
| `__cosf(x)` | cos x |
| `__sincosf(x,sptr,cptr)` | *sptr = sin x, cptr = cos x |
| `__tanf(x)` | tg x |
| `__powf(x, y)` | $x^y$ |

Здесь `rn,rz,ru,rd` - способы округления, например, `__fadd_rn` округляет сумму к ближайшему, `__fadd_rz` – к нулю, `__fadd_ru` – до большего, `__fadd_rd` – до меньшего.

Список встроенных функций для работы с типом double несколько меньше:

| **Функция** | **Операция** |
|---|---|
| `__dadd_[rn,rz,ru,rd](x,y),` | x + y |
| `__dsub_[rn,rz,ru,rd](x,y),` | x – y |
| `__dmul_[rn,rz,ru,rd](x,y),` | x * y |
| `__fma_[rn,rz,ru,rd](x,y,z)` | x * y + z |
| `__drcp_[rn,rz,ru,rd](x)` | 1 / x |
| `__dsqrt_[rn,rz,ru,rd](x)` | $\sqrt{x}$ |
| `__ddiv_[rn,rz,ru,rd](x,y)` | x / y |

## Практическая работа

1. Используя другие функции вместо синуса (__expf` (e^x), __exp10f` (10^x) и т. д.), рассчитайте относительную ошибку (отношение абсолютной ошибки к значению функции).

2. Напишите программу, реализующую скалярное произведение двух векторов.

## Контрольные вопросы

1. Как организовать обращение к элементу массива в теле ядра?
2. Какие основные этапы работы с векторами можно выделить?
3. Какие встроенные (быстрые) математические функции есть на GPU? В чем отличие их от обычных функций?